P0228R3
unique_function: a move-only std::function

Published Proposal, 2019-01-09

This version:
http://wg21.link/p0228r3

Authors:
Ryan McDougall <mcdougall.ryan@gmail.com>
Matt Calabrese <metaprogrammingtheworld@gmail.com>

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a conservative, move-only equivalent of `std::function`.

Table of Contents

§ 1. Brief History

This paper started as a proposal by David Krauss, N4543[1], from 2015 and there has been an open issue in the LEWG bugzilla requesting such a facility since 2014[2].

Since then, the paper has gone through 4 revisions and has been considered in small groups in LEWG multiple times. Gradual feedback has led to the conservative proposal seen here. The most-recent draft prior to this was a late-paper written and presented by Ryan McDougall in LEWGI in San Diego[3]. It included multiple references to implementations of move-only functions and made a strong case for the importance of a move-only form of `std::function`.

Feebdack given was encouragement for targetting C++20.

§ 2. Overview

This conservative `unique_function` is intended to be the same as `std::function`, with the exceptions of the following:

1. It is move-only.
2. It does not have the const-correctness bug of `std::function` detailed in n4348.[4]

3. It provides minimal support for cv/ref qualified function types.
4. It does not have the `target_type` and `target` accessors (direction requested by users and implementors).

§ 3. Summary of Changes

The following is not formal wording. It is an explanation of how this template and its specializations would differ from the specification of `std::function` relative to the WP from the most recent mailing, N4778.[5]

Add (for simplicity, class-definitions are not inlined here):

[func.wrap.func]

```
template<class Sig> class unique_function; // not defined

template<class R, class... ArgTypes>
  class unique_function<R(ArgTypes...)>;

template<class R, class... ArgTypes>
  class unique_function<R(ArgTypes...) const>;

template<class R, class... ArgTypes>
  class unique_function<R(ArgTypes...)&&>;
```

In [func.wrap.func.con]

Do not provide a copy constructor:

> ~~`unique_function(const unique_function& f);`~~

Do not provide a copy-assigment operator:

> ~~`unique_function& operator=(const unique_function& f);`~~

In [func.wrap.func] specify the following wording for aid in describing the provided `unique_function` partial specializations.

```
Let QUAL_OPT be an exposition-only macro defined to be a textual representation
of the qualifiers of the function type parameter of unique_function.

[Note:
  For unique_function<void() const>
    - QUAL_OPT is const

  For unique_function<void() &&>
    - QUAL_OPT is &&

  For unique_function<void()>
    - QUAL_OPT is
]

Let QUAL_OPT_REF be an exposition-only macro defined in the following manner:
    - If the function type parameter of unique_function is reference
      qualified, let QUAL_OPT_REF be defined as QUAL_OPT.

    - Otherwise, let QUAL_OPT_REF be defined as QUAL_OPT&.

[Note:
  For unique_function<void() const>
    - QUAL_OPT_REF is const&

  For unique_function<void() &&>
    - QUAL_OPT_REF is &&

  For unique_function<void()>
    - QUAL_OPT_REF is &
]
```

Update the function signature of `operator()` to match the template parameter exactly, and invoke the contained Callable with the correct cv qualification and value category (this prevents duplicating the const-correctness issues of std::function).

In [func.wrap.func.inv]

> `R operator()(ArgTypes... args)`~~`const`~~ `QUAL_OPT`;
>
> *Returns:* `INVOKE<R>( static_cast<remove_cvref_t<decltype(f)> QUAL_OPT_REF>( f ) , std::forward<ArgTypes>(args)...)` , where f is the unqualified target object of `*this` ,
>
> *Throws:* `bad_function_call` if `!*this` ; otherwise, any exception thrown by the wrapped callable object.

In [func.wrap.func.con] regarding the constructor taking a Callable, have a movability requirement instead of a copyability requirement and also require the correct kind of Callable:

> `template<class F> unique_function(F f)`
>
> *Requires:* F shall be ~~Cpp17CopyConstructible~~ Cpp17MoveConstructible
>
> *Remarks:* This constructor shall not participate in overload resolution unless
>
> `decay_t<F>` `QUAL_OPT_REF` is ~~Lvalue~~ Callable for argument types `ArgTypes...` and return type `R` .

Do the same for the converting assignment:

> `template<class F> unique_function& operator=(F&& f);`
>
> *Effects:* As if by: `unique_function(std::forward<F>(f)).swap(*this);`
>
> *Returns:* *this.
>
> *Remarks:* This assignment operator shall not participate in overload resolution unless `decay_t<F>` `QUAL_OPT_REF` is ~~Lvalue~~ Callable for argument types `ArgTypes...` and return type `R` .

Additionally, we suggest not including `target` and `target_type` , while being open to the possibility of proposing it for C++23:

```
const type_info& target_type() const noexcept;

template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
```

4. Implementation Experience

There are many implementations of a move-only `std::function` with a design that is similar to this. What is presented is a conservative subset of those implementations.

Previous revisions of this paper have included publicly accessible move-only function implementations, notably including implementations in HPX, Folly, and LLVM.

5. Suggested Polls

Proposal as-is?

Proposal with `target` and `target_type` *not* removed?

Proposal without the && specialization?

Proposal without the const specialization (no way to invoke a `const unique_function<void()>`).

Proposal with a more complete set of cv/ref specializations?

Name bikeshedding:

- reserve generalized terms like "func" et al for future types?
- prefer "move only" over "unique" nomenclature?
- such as "mfunction" or "mofunction"?

6. References

[1]: David Krauss: N4543 "A polymorphic wrapper for all Callable objects" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4543.pdf

[2]: Geoffrey Romer: "Bug 34 - Need type-erased wrappers for move-only callable objects" https://issues.isocpp.org/show_bug.cgi?id=34

[3]: Ryan McDougall: P0288R2 "The Need for std::unique_function" https://wg21.link/p0288r2

[4]: Geoffrey Romer: N4348 "Making std::function safe for concurrency" www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4348.html

[5]: Richard Smith: N4778 "Working Draft, Standard for Programming Language C++" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf